

The AI Coding Starter Guide

How to use Claude Code or Codex to turn a real request into working software—without blindly trusting the AI.

By Abhishek Raj

Built from 208 real coding-agent sessions at a live Indian startup doing about \$10M ARR with 100,000+ paying customers.

UNDERSTAND

DELEGATE

VERIFY + SHIP

You do not need magic prompts.

This guide comes from real work: 208 sessions with coding AI, 1,866 human messages, 667 code changes, and 200 pull requests at a live Indian startup doing about \$10M in yearly revenue with 100,000+ paying customers.

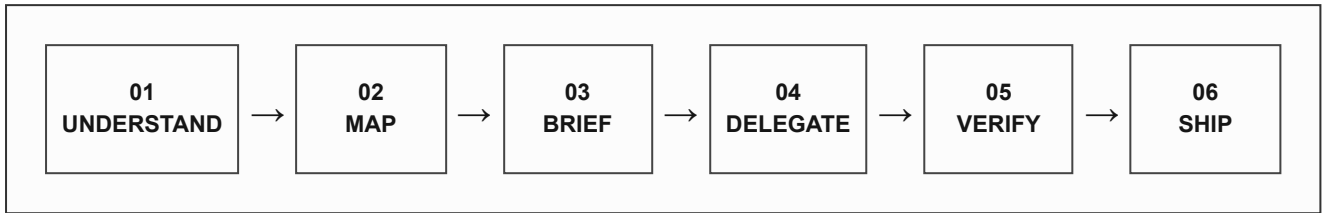
WHAT THE RECORD SHOWED	WHAT IT MEANS
67 characters median prompt length	Most useful steering was short; the agent already had durable context.
21% of prompts carried screenshots	The screenshot, portal response, log, or number was often the real bug report.
4 files median PR size	Small slices were fast to understand, verify, merge, and reverse.
133 memory files written	Reality outlived the context window instead of being reconstructed from chat.

The thesis: the leverage was not longer prompts. It was feeding the agent reality—the business rule, precedent, raw evidence, expected number—and checking the result against reality.

PRACTICE THIS

Tiny dictionary: branch = a separate version of code. Worktree = a separate folder for a branch. Test = a small computer check. Log = the app's diary of what happened. Diff = the list of code changes.

A simple loop: start right → understand → explain → build → check → share → remember



AI can make a change that looks good but is still wrong: it may be in the wrong folder, use the wrong rule, or solve a problem nobody has. Give it the right facts before you let it code.

STEP	WHAT YOU DO	HOW YOU KNOW YOU ARE READY
Start right	Check the right folder, branch, and app.	You know which code real users are using.
Understand	Write who needs help, what rule matters, and what can go wrong.	You can explain the problem in normal words.
Explain	Show AI a similar working feature, clear limits, and proof.	The job is small and clear.
Check + share	Look at the screen, network, logs, and data after the change.	You can stop or undo the change if needed.
Remember	Write down what really happened.	Next time, you do not start from zero.

PRACTICE THIS

At your next agent checkpoint, name the stage you are in. Do not let an implementation task begin until the Understand, Map, and Brief exit conditions are true.

First, make sure you are in the right project and branch.

A **branch** is a separate version of code. A **worktree** is a separate folder for that version. If you look at an old branch, you can give AI a perfect task and still get the wrong answer.

FIRST ACTION, EVERY TIME

01

Fetch the remote base

Do not treat a local branch called prod as production.

02

Name the worktree

One task, one worktree. Parallel agents do not share uncommitted edits.

03

Read production explicitly

Inspect the remote base when behavior must match a live path.

04

Locate the real environment

Confirm repository, service, worker, and data source before diagnosis.

05

Record the base

Put branch and commit in the task brief or memory entry.

06

Only then investigate

Now the agent has a trustworthy reality to inspect.

Hard-won rule: never commit from a worktree while an agent is editing it. A broad `git add -A` can sweep up its partial work; wait for the owner or let that agent commit its slice.

PRACTICE THIS

Before your next investigation, write down: remote base, worktree path, service name, and environment. If any field is uncertain, resolve it before reading code.

A simple note before you ask AI to code.

Fill this in before asking an agent to change anything. Start with who is eligible to enter the flow and why; unknowns stay visible rather than becoming the agent's assumptions.

```
# Business-gate brief

## Gate + user decision
Only [eligible user / entity] may [action] when [condition], because [business reason].

## Product end state
The user should be able to [observable outcome] in [time / UX budget].

## Current workflow
Today they [steps, tools, handoffs]. The friction is [cost / delay / error].

## Desired outcome
We will [observable behavior]. We will not [explicit non-goal].

## Failure cost
False approval causes: [impact].
False rejection causes: [impact].

## Precedent to match
[existing screen / service / workflow that already does this]

## Known rules
- [rule + source of truth]
- [rule + source of truth]

## Unknowns to resolve
- [question + owner]

## Success signal
[metric or observable user result]
```

Filled example: “Only customers who finish their profile can receive a personalised plan. Missing information goes to the support team, not a dead end. A support person’s change always wins.”

PRACTICE THIS

Create a file called docs/features/[feature].md and complete this brief. Leave unknowns visible; do not invent an answer for the agent.

Draw what happens, including where things can go wrong.

A feature is not only code. It also has people, data, screens, and outside tools. Draw the handoffs first. This helps you see where someone can get stuck.

CUSTOMER	Fills a sign-up form or sends a request
APP	Checks the details → chooses the next step
SUPPORT TEAM	Looks at unusual requests; can correct a choice
CUSTOMER	Gets a clear next step or a helpful message

MARK THESE FOUR THINGS ON EVERY MAP

Step changes

New request → checked → ready / needs help / blocked → customer told.

People in charge

Who can correct details? Who can change the app's choice?

Outside tools

Email service, file reader, payment tool, message sender.

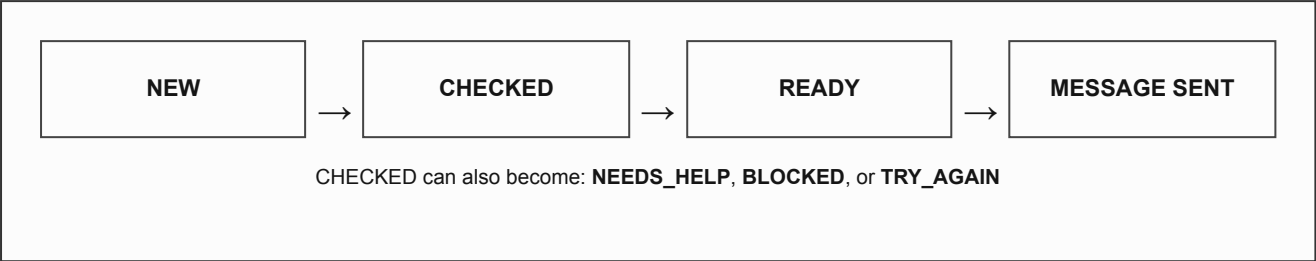
What can go wrong

Slow service, missing detail, same request twice, unsupported file.

PRACTICE THIS

Draw the current workflow first—not the ideal future one. Circle every handoff, spreadsheet, inbox, retry, and manual correction.

List the steps an item is allowed to take.



STEP	WHAT CAN HAPPEN NEXT	RULE	WHO DOES IT
NEW	CHECKED, TRY_AGAIN	All needed details arrive	App
CHECKED	READY, NEEDS_HELP, BLOCKED	App saves why it made the choice	App
NEEDS_HELP	READY, BLOCKED	A named person writes why	Support team
MESSAGE SENT	—	Do not send the same message twice	App

Why this matters for agents: state transitions turn vague “handle exceptions” language into code-level constraints an agent can test.

PRACTICE THIS
Write one invariant in the form: ‘The system must never ____.’ Turn it into a test before asking an agent to implement the flow.

Do not throw files at AI. Tell it the few things that matter.

More tokens do not automatically mean better work. Give the agent the smallest set of facts needed for its next correct decision: goal, gate, existing precedent, system constraints, relevant code, raw evidence, and behavior checks.

NOT HELPFUL	HELPFUL
"Make sign-up better. Look around."	"Follow the current sign-up path. Do not change anything yet. Tell me which pages, files, and checks are used."
"Use how we already build things."	"Put the checking rule in the same place as the existing sign-up checks. Do not overwrite a support person's choice."
"Handle missing details."	"If a needed detail is missing, ask for help. If it is clearly wrong, show a clear message. A support person's choice always wins."

Intent What outcome changes?	Ground truth Which rules and files are authoritative?	Verification What behavior proves it works?
--	---	---

Name the precedent: "Match X" is usually stronger than a page of invented architecture. It prevents a second validator, date helper, API client, or flow from appearing beside one that already works.

PRACTICE THIS

Replace one vague phrase in your next prompt ('handle errors', 'follow our architecture') with a named failure mode or a named code boundary.

Your “here is what is going on” note.

```
# CONTEXT – New customer sign-up

## Goal
Help new customers finish sign-up quickly while keeping people in
charge of unusual requests.

## Architecture
Sign-up form → sign-up service → simple checks → support queue
→ customer message.

## Precedent
Match `src/support/manual-choice.ts`: a support person’s choice always wins.

## Relevant files
- src/signup/signup-service.ts
- src/signup/check-details.ts
- src/signup/status.ts
- tests/signup/check-details.test.ts

## Constraints
- Never overwrite a support person’s correction.
- Save a short note about every app choice.
- Never send the same message twice.
- No new dependency without approval.

## Acceptance criteria
- Complete details: READY.
- Missing detail: NEEDS_HELP, not BLOCKED.
- Clearly wrong detail: BLOCKED with a reason.
- A person’s choice stays after the app checks again.

## Out of scope
Changing profile details automatically, redesigning the form, or changing
messages.
```

Store it: use `docs/features/signup-checks.md`, a pull-request description, or a small task file. A person should be able to read it too.

PRACTICE THIS

Copy this packet into your repository. Delete every line that is not true; fill every bracket from code, a schema, or the workflow owner—not from memory.

Ask AI to look around before it changes code.

The first request should reduce uncertainty, not create a diff. When the problem is not yet understood, use diagnosis-only mode: ask what is wrong, ask for evidence, and explicitly forbid edits.

```
You are investigating a feature. Do not modify files.
```

1. Trace a sign-up request from the form to its final result.
2. List the files that own the checks, status changes, notes, and customer message.
3. Find tests that cover each path.
4. Identify risks, especially sending two messages or overwriting a support person's choice.
5. Propose a minimal plan. For each step, name the file and test.

```
Return: findings, unknowns, plan, and what you deliberately will not change.
```

GIVE BUGS AS FALSIFIABLE EVIDENCE

- ☐ Attach the artefact: screenshot, raw portal error, log line, or response body.
- ☐ Name what happened and the exact value or behavior that should have happened.
- ☐ Withhold your diagnosis until the agent has inspected the evidence.
- ☐ On the second report, add a working sibling, a narrower value, or a new log—not the same complaint.

PRACTICE THIS

Run the investigation prompt without edit permission. Compare the returned plan against the repository yourself before authorizing any change.

A small, clear change request.

Once investigation is sound, convert the work into a bounded contract. This is what you delegate.

```
# Implement: sign-up detail checks
```

Make the smallest change that satisfies the acceptance criteria below.

Change

Add detail checks in `src/signup/check-details.ts`. Return one clear answer: READY, NEEDS_HELP, or BLOCKED with a short reason.

Must preserve

- A support person's choice always wins.
- Existing "do not send twice" behavior.
- A short saved note for every automatic choice.

Acceptance tests

1. complete details → READY
2. missing detail → NEEDS_HELP
3. clearly wrong detail → BLOCKED / WRONG_DETAIL
4. person's choice + wrong detail → person's choice stays

Boundaries

Do not change the database shape, form page, or message sender.
Do not add packages.

Deliverable

Implementation, focused tests, and a short explanation of changes and remaining risks.

PRACTICE THIS

Keep the first implementation brief smaller than you want. A useful first slice often changes one domain rule and one focused test file.

Do not ask AI to do everything in one go.

Long autonomous runs are useful only when each checkpoint has a small, observable output. A good workflow preserves your ability to redirect before the wrong assumption reaches five files.

01 Investigate No edits. Ask for paths, behavior, tests, risks.	02 Plan Review file-by-file plan and cut scope.
03 Implement One bounded slice; require tests.	04 Self-review Ask agent to criticize its own diff against criteria.
05 Adversarial review Fresh context: find what could break.	06 Run + inspect Tests, diff, logs, actual user journey.

Rule of thumb: you should be able to explain every changed file in one sentence. If you cannot, the batch was too large.

The exact command syntax differs between tools. The operating model does not: plan first, keep scope explicit, demand evidence, and review every consequential change.

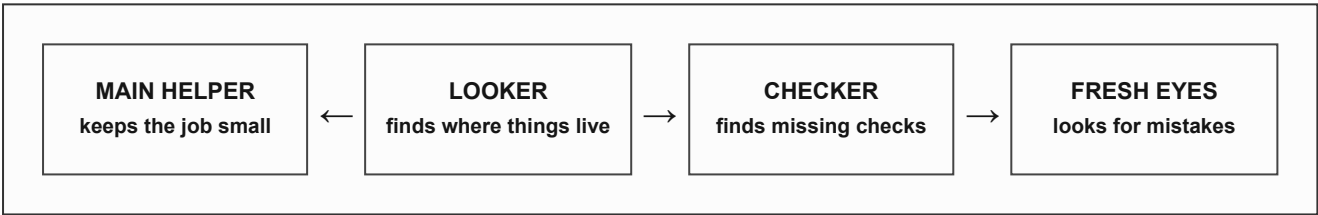
PRACTICE THIS

Put a hard pause between plan and implementation. The pause is where you remove assumptions, cut scope, and make the diff reviewable.

Let helpers look around. Do not let them fight over the same code.

Sometimes you can use more than one AI helper. Give each helper one small job with a clear name, like “find where login happens.” Do not ask three helpers to edit the same file. They will get in each other’s way.

GOOD HELPER JOB	WHY IT IS GOOD	BAD HELPER JOB
“Show me every place a sign-up request can change.”	It only looks and reports facts.	“All three of you build the same feature.”
“Find sample data and missing checks.”	You can easily check its list.	“All of you clean up this folder.”
“Look at these code changes for mistakes.”	It gets a fresh pair of eyes.	“Write it and say it is good in one step.”



Look first, build second: let one helper only look at folders, branches, pages, and rules. Then give one other helper a small coding job.

PRACTICE THIS
Before creating a subagent, finish this sentence: ‘Its output is valuable because I can verify it by ____.’ If you cannot, keep the task with the primary agent.

A simple job description for an AI helper.

ROLE: Repository researcher

MISSION

Trace how a sign-up request moves from the form to a customer message. Do not edit.

EVIDENCE REQUIRED

For every claim, provide: file path, symbol/function, and a one-line explanation of control flow.

QUESTIONS

1. Where are statuses defined and persisted?
2. Can checking again overwrite a support person's choice?
3. Which paths save a note about what happened?
4. Which checks cover trying again and duplicate messages?

OUTPUT FORMAT

```
## Findings
## Relevant files
## Risks / unknowns
## Suggested tests
```

STOP WHEN

You have answered the questions with evidence, or explicitly state what cannot be determined from the repository.

Good output

“`status.ts:moveSignup()` can send a message twice; it does not check the support person's choice.”

Weak output

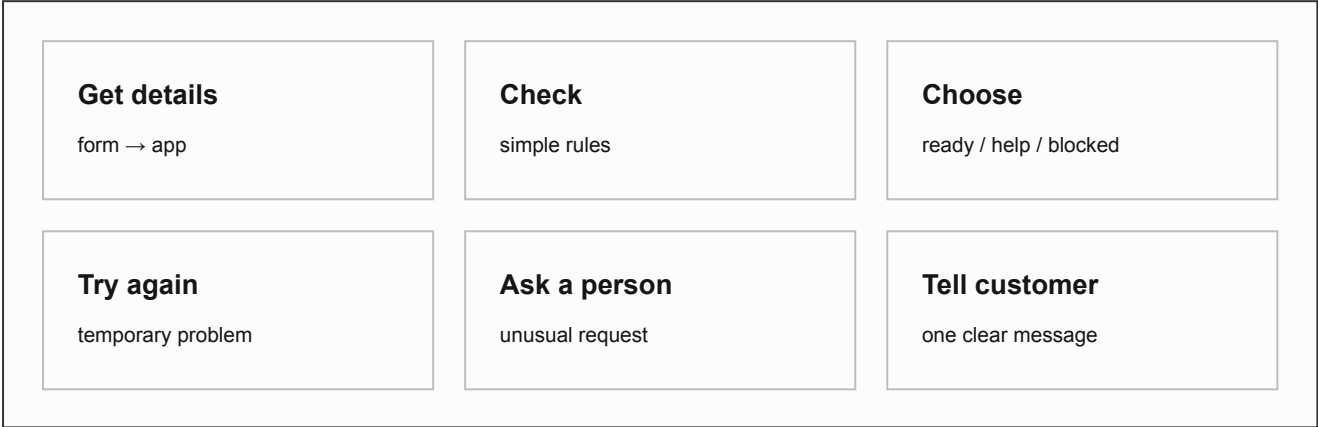
“The sign-up flow looks strong. Maybe add some checks.”

PRACTICE THIS

Give a researcher agent a stop condition. Endless exploration is not diligence; it is unbounded cost and an invitation for unsupported conclusions.

Use a simple map when work can branch, pause, or try again.

For a very simple job, AI can do one thing and finish. For a tricky job, draw the path. Show what happens when it works, when it fails, when it needs a person, and when it should try again.



DRAW A MAP WHEN...	KEEP IT SIMPLE WHEN...
Different people handle different paths.	One request always gives one answer.
Trying again could send something twice.	If it fails, you can safely show an error.
A person needs to approve before it continues.	Nothing needs to be saved between steps.

PRACTICE THIS

Sketch every branch that has a different owner, a retry, or an irreversible action. Those branches deserve explicit states—not a hidden chain of if-statements.

Write the rules of a tricky flow before coding it.

STEP: check_signup

Input: customer_id, form_details, support_choice

Output: next_step, short_reason, saved_note

TRANSITIONS

- complete → ready
- missing needed detail → needs_help
- clearly wrong detail → blocked
- message service is slow → try_again (max 3 attempts)
- support_choice present → use the person's choice

INVARIANTS

- A customer gets a message only when ready.
- Every step saves a short note.
- Trying again cannot send a second message.
- The app never replaces a person's choice.

OBSERVABILITY

- Count: how many requests reach each next step
- Count: how many need to try again
- Log: customer_id, old_step, new_step, short_reason
- Alert: too many requests waiting for more than 15 minutes

Simple rule: write one sentence that says what the app must never do. Example: "Never send the same customer message twice." That is better than saying "make it reliable."

PRACTICE THIS

Choose one metric and one alert before implementation. If a stuck or unsafe transition cannot be detected, it is not ready to automate.

Do not only read the code. Check what the user will see.

A test is a small computer check. It is useful, but it is not the whole answer. Also open the real screen, click through the flow, and look at the data. Do both: computer checks and real-life checks.

Small check
Does one rule work?

Parts together
Do the app pieces talk to each other?

Real journey
Can a person upload, review, and finish?

TRY THIS	YOU SHOULD SEE	THIS STOPS
Complete form	The customer can continue and a note is saved	A normal flow breaking
Missing detail	It goes to a person to help	A good customer being blocked
Clearly wrong detail	A clear message explains what to fix	Bad data passing quietly
A person changes it	Their change stays after the app runs again	The app overwriting a person
The message runs twice	Only one message is sent	Duplicate messages

What to trust most: the real website or outside system, then your saved data and logs, then the code, and lastly an AI saying “done.” For a web page, check the screen, network, and console; then reload it.

PRACTICE THIS
Ask the workflow owner for the example they worry about most. Add it to the matrix exactly as they describe it, even if the input is messy.

A “what good looks like” table.

Write this table before you ask AI to code. It gives you a simple picture of what “working” means.

TRY THIS	START WITH	YOU SHOULD SEE	TYPE OF CHECK
Complete form	Name and email are present	next_step=READY	Small + parts together
Missing detail	`phone: null`	next_step=NEEDS_HELP	Small + real journey
Wrong format	`email: "hello"`	next_step=BLOCKED; clear reason	Small check
Person fixes it	Support choice exists	choice stays after checking again	Parts together
Same message twice	Same request id twice	one saved note + one message	Parts together
Outside service is slow	Message sender times out	TRY_AGAIN; no message yet	Parts together

Tell AI: “Make these cases work. Then tell me which important cases I may still be missing. Do not say it is tested unless you can show me.”

PRACTICE THIS

Take one real support problem and add it to this table. If your app made that mistake once, make sure the next version does not repeat it.

Ask a fresh AI chat to try to break the change.

The AI that wrote the code may miss its own mistake. Open a fresh AI chat and ask it to look for problems. This is extra helpful for login rules, repeated actions, lost data, and anything a user cannot easily undo.

```
Review this diff as a production engineer. Do not rewrite it.
```

```
The feature must: preserve manual overrides, emit audit entries,  
avoid duplicate messages, and send missing details to the support team.
```

```
Check for:
```

- ```
- behavior that violates an acceptance criterion
```
- ```
- missing or misleading tests
```
- ```
- unauthorized state transitions
```
- ```
- race conditions / retries / duplicate events
```
- ```
- sensitive data in logs
```
- ```
- backwards compatibility risk
```

```
For every finding, cite file + line, severity, failure scenario, and  
the smallest credible remediation. If no finding, state what you could  
not verify.
```

DO NOT ACCEPT THIS REVIEW OUTPUT

“Looks good,” “consider adding error handling,” or a long list without file evidence. The agent must connect each concern to a real execution path.

PRACTICE THIS

Review a diff from a clean conversation or a second agent. Fresh context is an inexpensive way to challenge implementation anchoring.

A safety checklist before you share the change.

QUESTION	EVIDENCE TO DEMAND
Can an incorrect state transition happen?	State guard, test, and audit record.
Can a retry duplicate a side effect?	Idempotency key or unique constraint + integration test.
Can a human correction be lost?	Explicit precedence rule + reprocessing test.
Can an external outage create a bad decision?	Timeout policy, retry path, and alert.
Can sensitive data leak?	Log inspection; redaction or field allow-list.
Can we see and reverse the change?	Metrics, dashboard/query, flag or rollback plan.

Big problem

Lost data, charging twice, showing private data, or something you cannot undo.

Smaller problem

Sending someone to the wrong place, missing a helpful note, or a visible error they can recover from.

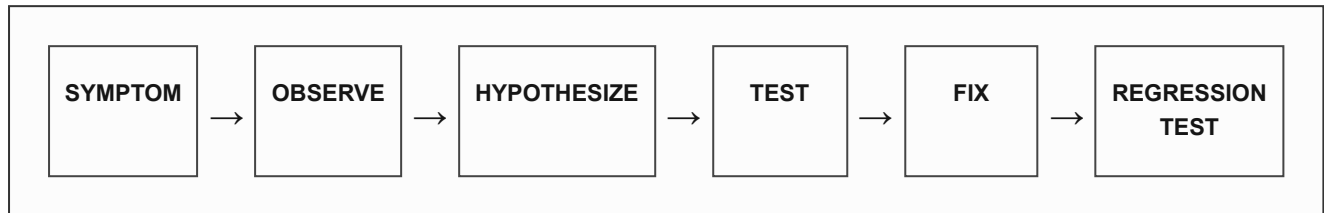
Make it a habit: review the release path, not only the diff. Code can be locally correct and still fail through migration order, missing configuration, or queue behavior.

PRACTICE THIS

For the most expensive failure mode, write a one-line answer to each rubric question. Missing evidence is a release decision, not a documentation issue.

Use facts first. Do not let AI guess a fix.

AI is good at making believable guesses. Do not let a believable guess become a fix. First collect clues: what the user saw, the error message, and what number or result should have appeared.



EXAMPLE: SIGN-UP MESSAGES ARE STUCK

Weak request

“Why are messages stuck? Fix it.”

Strong request

“Look at the logs and code for three customer requests. Make a timeline. Do not change code until you find what they all have in common.”

Rule: a fix is not complete until the original symptom can no longer recur under a test, monitor, or invariant.

PRACTICE THIS

When debugging, first request a timeline—not a fix. Facts arranged in time often expose a retry loop, race condition, or configuration boundary immediately.

A note for finding a bug step by step.

```
# Problem: customer messages stuck in TRY_AGAIN

## Symptom
What happened? When? Which users / records are affected?

## Timeline
time | customer_id | old_step | event | new_step | request_id

## Evidence
- logs:
- metrics:
- recent deploy / configuration change:
- dependency health:

## Hypotheses (ranked)
1. [claim] – evidence for / against – next observation
2. [claim] – evidence for / against – next observation

## Fix
Smallest change that removes the proven cause:

## Regression proof
Test added:
Monitor / alert added:
Runbook update:
```

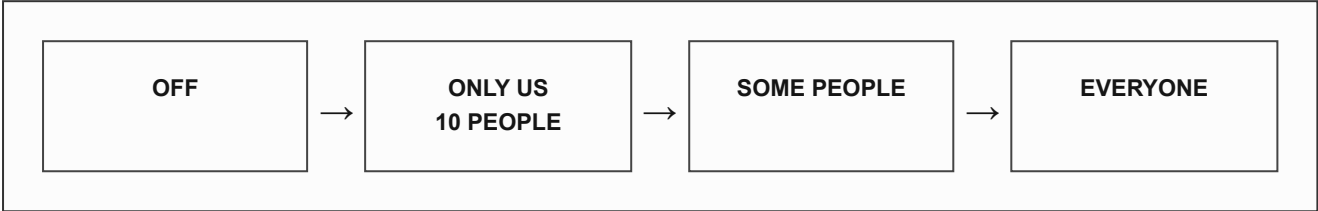
Useful agent role: ask it to normalize logs into a timeline, find common fields among failing events, and identify code paths. You retain the decision about cause and fix.

PRACTICE THIS

Save this worksheet with the incident. The next engineer should be able to see the evidence that eliminated each wrong hypothesis.

Let a few people use the change before everyone does.

“The code is live” does not mean it is good. Decide before you share it: what will you watch, when will you pause, and how will you turn it off if it hurts users?



BEFORE SHARING	WHILE SHARING	AFTER SHARING
Off switch available Simple dashboard ready Undo steps written down	Start with a few people Watch counts and errors Look at real saved notes	Compare “needs help” numbers Check for duplicate messages Talk to a support person

Simple safety rule: try it with no real customers first, then 2, then 10, then everyone. If too many people get stuck, turn it off and send them to the old helpful path.

PRACTICE THIS
Before you share a change, write down: who will watch it, what looks wrong, when you will stop it, and how you will turn it off. If you cannot answer those, wait.

A small plan for putting a change live.

```
# Release: sign-up detail checks

## Preconditions
- migration: none
- flag: signup_detail_checks
- dashboard/query: next steps by reason + message count
- owner on call: [name]

## Phased rollout
1. Run against 10 made-up customer requests; compare to a support person's choice.
2. Enable for internal test account.
3. Enable for 10% of new sign-ups for 60 minutes.
4. Expand only if success criteria are met.

## Stop / rollback conditions
- any duplicate message
- missing detail blocks a customer instead of asking for help
- unexplained error rate above baseline

## Recovery
Disable flag. Send records in affected time window to manual review.
Use audit events to reconcile outcomes.

## Success after 7 days
Manual review reduced without an increase in corrected decisions.
```

PRACTICE THIS

Write the rollback steps while the change is still small. A rollback plan that depends on remembering the implementation is not a plan.

Write down what surprised you so nobody has to learn it twice.

Production changes teach you which assumptions were false. Capture that learning while the details are fresh, then turn it into a rule, fixture, test, runbook entry, or context file. In the field record, 133 durable memory files made this work survive context windows and agent changes.

What surprised us?

Which workflow detail was missing from the ticket?

What did the agent assume?

Which assumption was plausible but wrong?

What signal was missing?

Which log, metric, or audit detail would have shortened debugging?

What becomes reusable?

Which test fixture or brief template should be kept?

Memory rule: record only what was observed. Label inference, assumption, and untested claims as unverified. Keep the hypothesis you killed, the baseline failure count, the blast radius, and the external verdict verbatim.

If you type an instruction twice, move it into the right durable layer: workspace instructions for standing defaults, a skill for domain maps, an agent definition for role behavior, or memory for observed facts.

PRACTICE THIS

After your next release, add one durable artifact: a fixture, a runbook step, an architecture note, a log field, or a sharper acceptance criterion.

Care about the person using it, not only the code task.

A useful AI-native engineer is not someone who produces the most code per hour. They reduce the time from unclear problem to trusted outcome.

TICKET MINDSET	FDE MINDSET
"What should I build?"	"What decision or workflow should improve?"
"The requirements were incomplete."	"Here are the unknowns and the lowest-risk way to resolve them."
"The tests pass."	"The behavior is observable, safe to release, and works for the user."
"The agent implemented it."	"I can explain why this design is correct and what it cannot guarantee."

AI makes implementation cheaper. It makes judgment, discovery, verification, and ownership more visible.

The honest caveat: speed can hide weak review. The record contained zero explicit test requests in sampled sessions, many symptom-only re-reports, and long sessions that needed repeated compaction. Keep the fast empirical loop—but add a diff glance, targeted test, fresh review pass, and a work-in-progress limit.

PRACTICE THIS
During planning, ask 'What would make this outcome wrong for the user?' That question shifts the conversation from code output to ownership.

How to use this on your next small feature.

01

30 minutes: discovery

Write the decision brief. Speak to the workflow owner if one fact is uncertain.

02

30 minutes: repository trace

Let an agent investigate without edits. Verify its file map.

03

20 minutes: scope

Write acceptance criteria, invariants, and out-of-scope items.

04

45 minutes: implementation

Delegate one bounded slice. Keep every diff explainable.

05

30 minutes: adversarial review

Use a fresh context to attack assumptions, tests, and failure modes.

06

30 minutes: release design

Choose a rollout, signals, stop condition, and recovery path.

Do not optimize this schedule. It is a forcing function that keeps the highest-leverage thinking before the lowest-leverage typing. Adjust duration, not order.

PRACTICE THIS

Timebox the loop for one feature this week. The aim is not to spend more time; it is to spend the first hour where errors are cheapest to catch.

Get better by making small things, not collecting tools.

WEEK	PRACTICE	PROOF OF WORK
1 — Understand	Choose one existing internal or personal workflow. Interview its user or reconstruct the current process.	Decision brief + workflow map.
2 — Context + plan	Use Claude Code or Codex to trace a code path without edits. Create a context packet and implementation brief.	Repository map + reviewed plan.
3 — Build + verify	Implement one bounded feature. Write the acceptance matrix before the agent writes tests.	Small PR + test evidence + review notes.
4 — Ship + learn	Release safely, or simulate a release with flags, metrics, and a rollback plan.	Release plan + retrospective.

RULES FOR THE MONTH

- ☐ Never ask an agent to “just build it” without a written acceptance standard.
- ☐ Never merge a consequential diff you cannot explain.
- ☐ For every bug, add a regression proof—not just a patch.
- ☐ Save your best briefs and fixtures in the repository.

PRACTICE THIS

Choose a feature small enough to release or simulate this month. Practice the full loop; isolated prompting practice does not build production judgment.

Use this before you say “done.”

ESTABLISH + MAP

- ☐ Fresh remote base and correct worktree confirmed
- ☐ Named user, business gate, and decision
- ☐ Current workflow and precedent documented
- ☐ Human owner and override path clear
- ☐ States and external boundaries mapped

BRIEF + DELEGATE

- ☐ Raw evidence and expected value attached
- ☐ Acceptance criteria observable
- ☐ Out-of-scope work explicit
- ☐ Scout investigated before builder edits
- ☐ Tasks are independently reviewable

VERIFY

- ☐ Happy, edge, failure, and retry cases tested
- ☐ Manual decisions are preserved
- ☐ Fresh adversarial review completed
- ☐ Screen, network, logs, database, and diff inspected
- ☐ Baseline failures separated from new ones

SHIP + LEARN

- ☐ Dry run → 2 → 10 rollout exists
- ☐ Signals, stop condition, and recovery owner named
- ☐ Blast radius stated, including what does not change
- ☐ Customer data absent from pushed artifacts
- ☐ Observed learning written to durable context

The standard: not “the agent finished.” Not “the build passed.” A real outcome is understandable, testable, observable, and recoverable.

PRACTICE THIS

Print or pin this checklist beside your pull request. A checked box is not proof, but an unchecked box is a useful conversation starter.

Use AI to build faster. Keep thinking for yourself.

The durable advantage is not knowing a magic prompt or a particular tool. It is knowing how to discover a real problem, give implementation the right constraints, verify the result, and take responsibility for what reaches users.

Claude Code and Codex can make a capable engineer dramatically faster. They cannot make an unclear workflow clear, decide whether a business rule is right, notice an unspoken exception, or accept the consequences of a bad release.

That remains the work. It is also the opportunity.

Next action: choose one small production-adjacent workflow this week. Complete pages 5, 7, 9, 11, 18, and 24 before asking an agent to implement anything.

Abhishek Raj

AI-native engineering, browser automation, and production software.

PRACTICE THIS

Start with one genuine workflow and make one durable improvement. The habit of shipping reliable small changes compounds faster than collecting agent tricks.